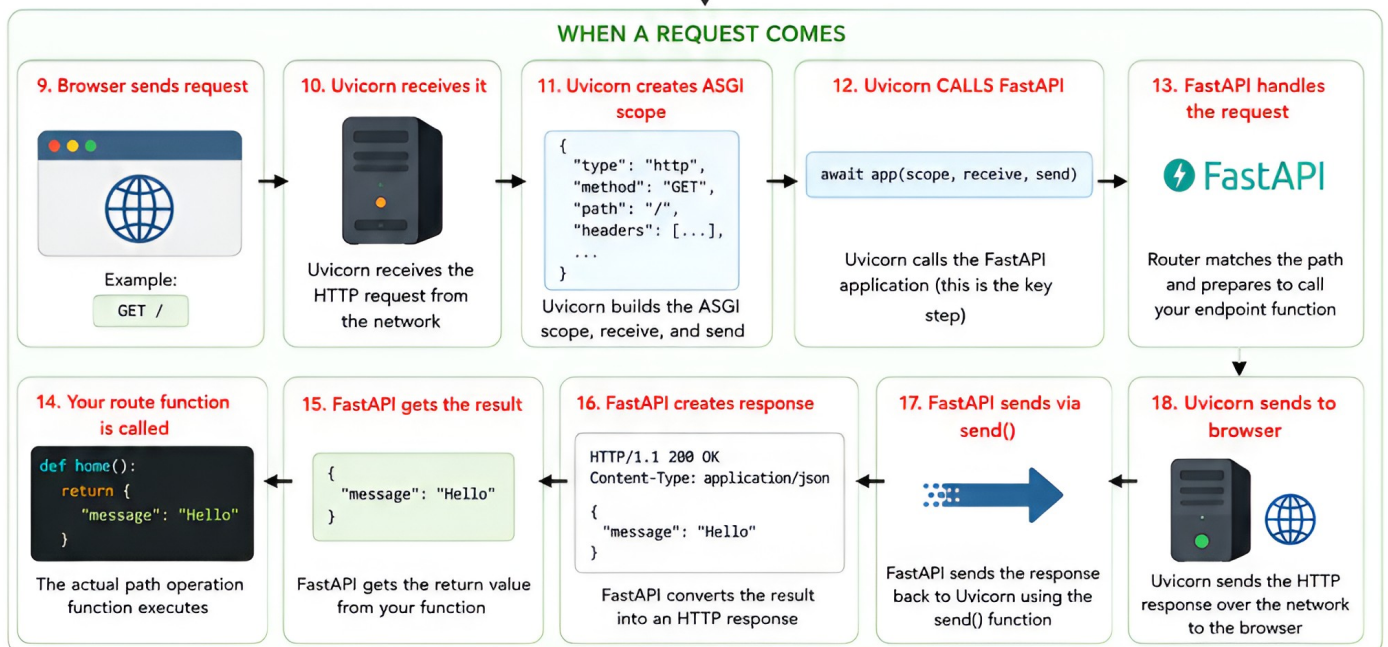
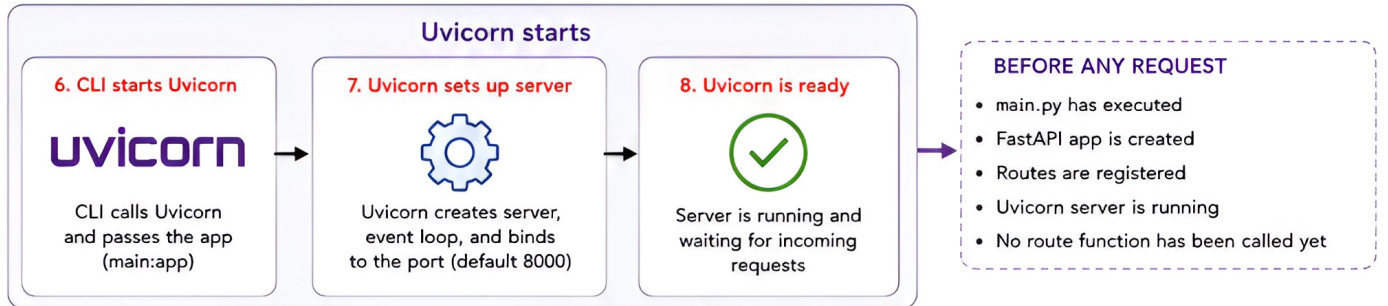
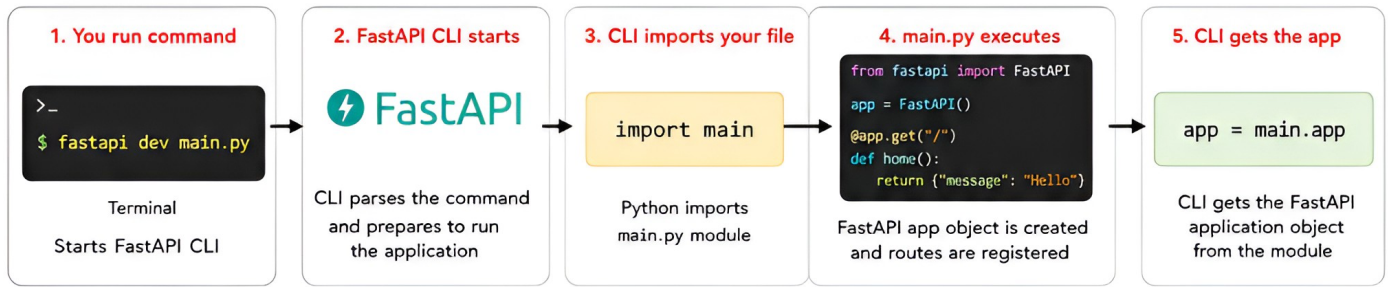


FASTAPI + UVICORN LIFE CYCLE (FROM START TO END)



IMPORTANT TIMELINE (REAL ORDER OF EXECUTION)

1	You run: fastapi dev main.py	Terminal
2	FastAPI CLI starts	FastAPI CLI
3	CLI imports your file	Python (import main)
4	main.py starts executing	main.py
5	FastAPI app object is created (app = FastAPI())	main.py
6	Routes are registered (@app.get, etc.)	main.py
7	main.py execution finishes	Python
8	CLI gets app (main.app)	FastAPI CLI
9	CLI starts Uvicorn	FastAPI CLI
10	Uvicorn files start executing (uvicorn.main)	uvicorn/ (inside-site-packages)
11	Uvicorn imports module and gets app (main:app)	uvicorn/config.py, importer.py
12	Uvicorn sets up server, event loop, socket, etc.	uvicorn/server.py
13	Uvicorn starts listening for requests	uvicorn/server.py
14	Browser sends HTTP request	Browser/Network
15	Uvicorn receives request and builds ASGI scope	uvicorn/protocols/http/h11_impl.py
16	Uvicorn calls FastAPI app -> await app(scope, receive, send)	uvicorn/protocols/http/h11_impl.py
17	FastAPI handles request, routing, calls your function	fastapi/routing.py -> your code
18	Response flows back: FastAPI -> Uvicorn -> Browser	fastapi -> uvicorn -> browser

STARTUP PHASE
(HAPPENS ONCE)

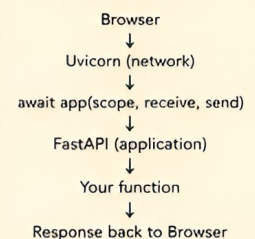
SERVER STARTUP PHASE
(HAPPENS ONCE)

REQUEST HANDLING PHASE
(REPEATS EVERY REQUEST)

KEY POINTS

- main.py runs only once at startup.
- Route functions run only when a request comes.
- Uvicorn is the server (handles network).
- FastAPI is the application (handles logic).
- The heart of the system: await app(scope, receive, send)

THE CORE FLOW



This diagram explains the **complete execution flow of a FastAPI application running on Uvicorn**, from the moment you type a command in the terminal until a browser receives a response.

The most important thing to understand is:

FastAPI itself is **not the web server**.

FastAPI is the **ASGI application**.

Uvicorn is the **ASGI server** that runs your app and handles networking.

Big Picture First

There are **3 major phases** in the diagram:

- 1. Startup Phase**

Happens once when you start the application.

- 2. Server Startup Phase**

Uvicorn initializes networking and prepares to accept requests.

- 3. Request Handling Phase**

Happens every time a client sends a request.

Example Application Used in the Diagram

The diagram assumes a file named `main.py`.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello"}
```

And you run:

```
fastapi dev main.py
```

PART 1 – STARTUP PHASE (HAPPENS ONCE)

Step 1 – You Run the Command

```
fastapi dev main.py
```

What happens?

- You are executing the **FastAPI CLI tool**
- The OS launches a Python process
- The CLI begins parsing your command

Here:

- `fastapi` → CLI executable
 - `dev` → development mode
 - `main.py` → your application file
-

Important Understanding

At this moment:

- Your route functions are NOT running
- No HTTP request exists
- Browser is not connected
- Uvicorn server is not yet fully running

You only started the CLI.

Step 2 – FastAPI CLI Starts

The FastAPI CLI program begins execution.

Its responsibilities:

- Read command-line arguments
- Determine app location
- Configure development mode
- Prepare Uvicorn configuration
- Enable auto-reload (in dev mode)

Internally it does something conceptually similar to:

```
parse_command()  
locate_app()  
prepare_uvicorn()
```

Development Mode (`dev`)

`fastapi dev` usually enables:

- Auto reload
- Debugging support
- File watching

Meaning:

If you change code:

```
@app.get("/")  
def home():  
    return {"message": "Changed"}
```

the server reloads automatically.

Step 3 – CLI Imports Your File

This is EXTREMELY important.

The CLI now imports:

```
import main
```

This means Python executes the module.

Critical Python Concept – Import Executes the File

When Python imports a module:

```
import main
```

Python executes ALL top-level code inside `main.py`.

So this runs immediately:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello"}
```

VERY Important Understanding

The route function:

```
def home():
```

is NOT executed during import.

ONLY these things happen:

- Function object is created
 - Decorator executes
 - Route gets registered
-

Step 4 – `main.py` Executes

Now Python executes the file line by line.

Line 1

```
from fastapi import FastAPI
```

Python imports the FastAPI class.

No app yet.

Line 2

```
app = FastAPI()
```

This is VERY important.

A FastAPI application object is created.

Internally this object stores:

- Routes
- Middleware
- Dependency injection system
- Exception handlers
- Event handlers
- OpenAPI schema system
- Lifespan management

Think of it as:

```
app = {  
    routes: [],  
    middleware: [],  
    config: {},  
}
```

(not exact implementation)

Step 5 – Route Registration

Now this line executes:

```
@app.get("/")
```

This decorator executes IMMEDIATELY during import.

It registers the route.

Conceptually:

```
app.routes.append({  
    "path": "/",  
    "method": "GET",  
    "handler": home  
})
```

VERY IMPORTANT

Still:

```
def home():
```

has NOT executed.

Only the function reference is stored.

This is one of the most important FastAPI concepts.

After Import Completes

Now:

- `main.py` finished executing
- `app` exists
- Routes are registered

But:

- No request yet
- No route executed yet

Step 6 – CLI Gets the App

The CLI retrieves:

```
app = main.app
```

This is why many tutorials write:

```
uvicorn main:app
```

Meaning:

```
module:variable
```

So:

- `main` → Python module
- `app` → FastAPI object inside module

uvicorn `main:app` really mean?

CLI starts **Uvicorn** with app reference string(`main:app`).

`main:app` Means:

`main` → Python module (main.py file, at the root of the project)

`app` → variable inside that module

So **Uvicorn** interprets it like:

“Import `main.py`, then take the variable named `app` from it”.

PART 2 – UVICORN SERVER STARTUP PHASE

Step 7 – CLI Starts Uvicorn

Now the CLI launches Uvicorn.

Conceptually:

```
uvicorn.run(app)
```

or:

```
uvicorn main:app
```

What Is Uvicorn?

Uvicorn is:

- An ASGI server
- Handles networking
- Handles sockets
- Receives HTTP requests
- Sends HTTP responses

FastAPI itself does NOT listen to ports.

Uvicorn does.

ASGI (Very Important)

ASGI = Asynchronous Server Gateway Interface

It is a standard protocol between:

- Web server (Uvicorn)
- Python application (FastAPI)

Think:

```
Browser <-> Uvicorn <-> FastAPI
```

ASGI is the communication contract.

Step 8 – Uvicorn Sets Up the Server

Now Uvicorn initializes:

- Event loop
 - Socket
 - HTTP protocol parser
 - Async system
 - Port binding
-

Event Loop

For async execution.

Usually powered by:

- asyncio
- uvloop

It manages:

- Coroutines
 - Async tasks
 - Network I/O
-

Port Binding

By default:

```
127.0.0.1:8000
```

Uvicorn opens a socket and listens there.

Conceptually:

```
socket.bind(("127.0.0.1", 8000))  
socket.listen()
```

IMPORTANT

Now the server is waiting.

Nothing happens until a request arrives.

BEFORE ANY REQUEST

The diagram correctly says:

main.py Has Executed

TRUE.

Import already happened.

FastAPI App Is Created

TRUE.

app = FastAPI() already ran.

Routes Are Registered

TRUE.

Decorators already executed.

Uvicorn Server Is Running

TRUE.

Socket listening started.

No Route Function Has Been Called Yet

VERY IMPORTANT.

This still did NOT happen:

```
home ( )
```

PART 3 – REQUEST HANDLING PHASE (REPEATS FOR EVERY REQUEST)

Now a browser sends:

```
GET /
```

Step 9 – Browser Sends Request

Example:

```
GET / HTTP/1.1
Host: localhost:8000
```

Browser converts your URL into an HTTP request.

Step 10 – Uvicorn Receives the Request

Uvicorn accepts network data from the socket.

It parses:

- HTTP method
- Headers
- Path
- Body

Uvicorn handles low-level networking.

FastAPI does NOT directly read sockets.

Step 11 – Uvicorn Creates ASGI Scope

VERY IMPORTANT STEP.

Uvicorn converts the HTTP request into an ASGI format.

Example:

```
scope = {
    "type": "http",
    "method": "GET",
    "path": "/",
    "headers": [...],
}
```

What Is **scope**?

`scope` contains request metadata.

It is created ONCE per request.

Contains:

- Method
 - Path
 - Query string
 - Headers
 - Client info
 - Server info
-

ASGI Also Uses `receive` and `send`

ASGI applications receive 3 things:

```
await app(scope, receive, send)
```

Meaning of Each

1. `scope`

Request information.

2. `receive`

Function to receive incoming data.

Used for:

- Request body
 - Streaming uploads
 - WebSockets
-

3. send

Function to send response data back.

Step 12 – Uvicorn Calls FastAPI

THIS IS THE CORE STEP.

Uvicorn calls:

```
await app(scope, receive, send)
```

This is the ASGI protocol.

Critical Understanding

Your FastAPI app object is actually an ASGI callable.

Conceptually:

```
class FastAPI:
    async def __call__(self, scope, receive, send):
        ...
```

So Uvicorn treats FastAPI as a callable ASGI app.

The Entire Flow Depends on This

```
Uvicorn ---> await app(scope, receive, send)
```

This is the heart of FastAPI/Uvicorn interaction.

Step 13 – FastAPI Handles the Request

FastAPI now:

- Reads path
 - Matches route
 - Resolves dependencies
 - Validates parameters
 - Parses body
 - Prepares endpoint execution
-

Route Matching

FastAPI searches registered routes.

Example:

```
@app.get("/")
```

matches:

```
GET /
```

Step 14 – Your Route Function Executes

NOW this finally happens:

```
home()
```

This is the first time your function runs.

Important

The function runs PER REQUEST.

If 100 requests arrive:

```
home()
```

runs 100 times.

Step 15 – FastAPI Gets the Result

Your function returns:

```
{"message": "Hello"}
```

This is just a Python dictionary.

Not yet an HTTP response.

Step 16 – FastAPI Creates HTTP Response

FastAPI converts the dictionary into:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"message": "Hello"}
```

FastAPI Internally Does

- JSON serialization
 - Status code handling
 - Header creation
 - Response object creation
-

Step 17 – FastAPI Uses `send()`

FastAPI sends response data back through ASGI.

Conceptually:

```
await send({
    "type": "http.response.start",
    ...
})

await send({
    "type": "http.response.body",
    ...
})
```

VERY IMPORTANT

FastAPI does NOT directly write to the socket.

It sends ASGI messages to Uvicorn.

Uvicorn handles actual networking.

Step 18 – Uvicorn Sends Response to Browser

Uvicorn:

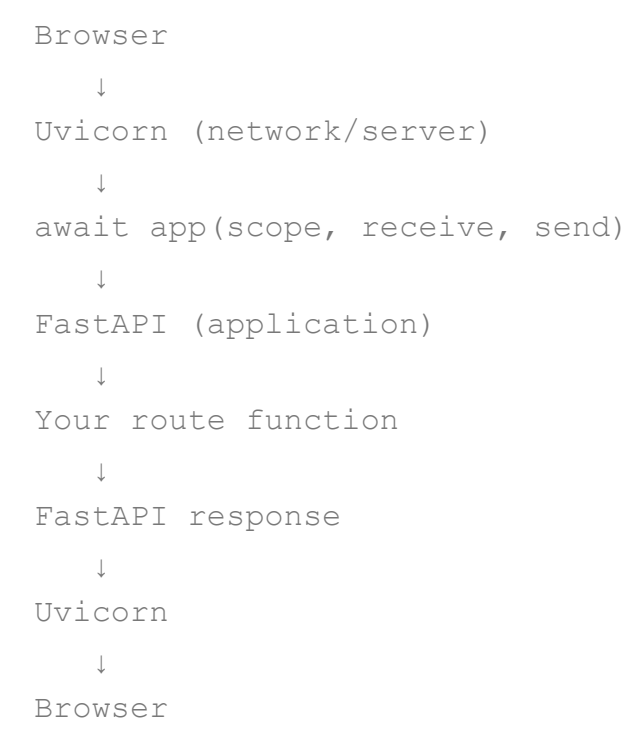
- Converts ASGI response to HTTP bytes
- Writes bytes to socket
- Browser receives response

Browser displays:

```
{"message": "Hello"}
```

CORE EXECUTION FLOW (MOST IMPORTANT)

The diagram summarizes:



This is the most important architecture understanding.

IMPORTANT DISTINCTION

Component	Responsibility
Browser	Sends HTTP request
Uvicorn	Networking/server
ASGI	Communication protocol
FastAPI	Application framework
Route Function	Business logic

Extremely Important Lifecycle Insight

Happens ONLY ONCE

These happen once at startup:

- Importing module
 - Creating app
 - Registering routes
 - Starting Uvicorn
 - Opening socket
-

Happens PER REQUEST

These repeat:

- Request received
 - Scope creation
 - Route matching
 - Function execution
 - Response generation
-

Common Beginner Misunderstanding

Many beginners think:

```
@app.get("/")
def home():
```

runs when the server starts.

FALSE.

Only the decorator executes at startup.

The function body runs only when a matching request arrives.

Another Important Internal Detail

FastAPI is built on:

- Starlette (web framework internals)
- Pydantic (validation)

So internally:

```
FastAPI -> Starlette -> ASGI
```

Uvicorn communicates with Starlette/FastAPI through ASGI.

About Async Endpoints

If you write:

```
@app.get("/")
async def home():
    return {"message": "Hello"}
```

then FastAPI awaits the coroutine.

Still same ASGI lifecycle.

Only difference:

- async execution
 - non-blocking I/O support
-

Real Internal Chain (Advanced View)

Actual internal call chain is approximately:

```
Browser
↓
TCP Socket
```

↓
Uvicorn
↓
HTTP Parser
↓
ASGI scope creation
↓
FastAPI.__call__()
↓
Starlette Router
↓
Dependency Injection
↓
Endpoint Function
↓
Response Object
↓
ASGI send()
↓
Uvicorn
↓
Socket
↓
Browser

One More Critical Concept – Why `main.py` Runs Only Once

Because Python caches imported modules.

After:

```
import main
```

Python stores it in:

```
sys.modules
```

So repeated requests do NOT re-import your app file.

Only endpoint functions execute repeatedly.

Final Mental Model

Think of FastAPI apps like this:

Startup Phase

```
Create application
Register routes
Start server
Wait...
```

Request Phase

```
Receive request
Find matching route
Run endpoint function
Create response
Send response
```

Most Important Line in Entire Diagram

This:

```
await app(scope, receive, send)
```

If you deeply understand this line, you understand:

- ASGI
- FastAPI lifecycle
- Uvicorn's role
- Request flow

- Response flow
- Separation of server vs application

Everything revolves around this call.